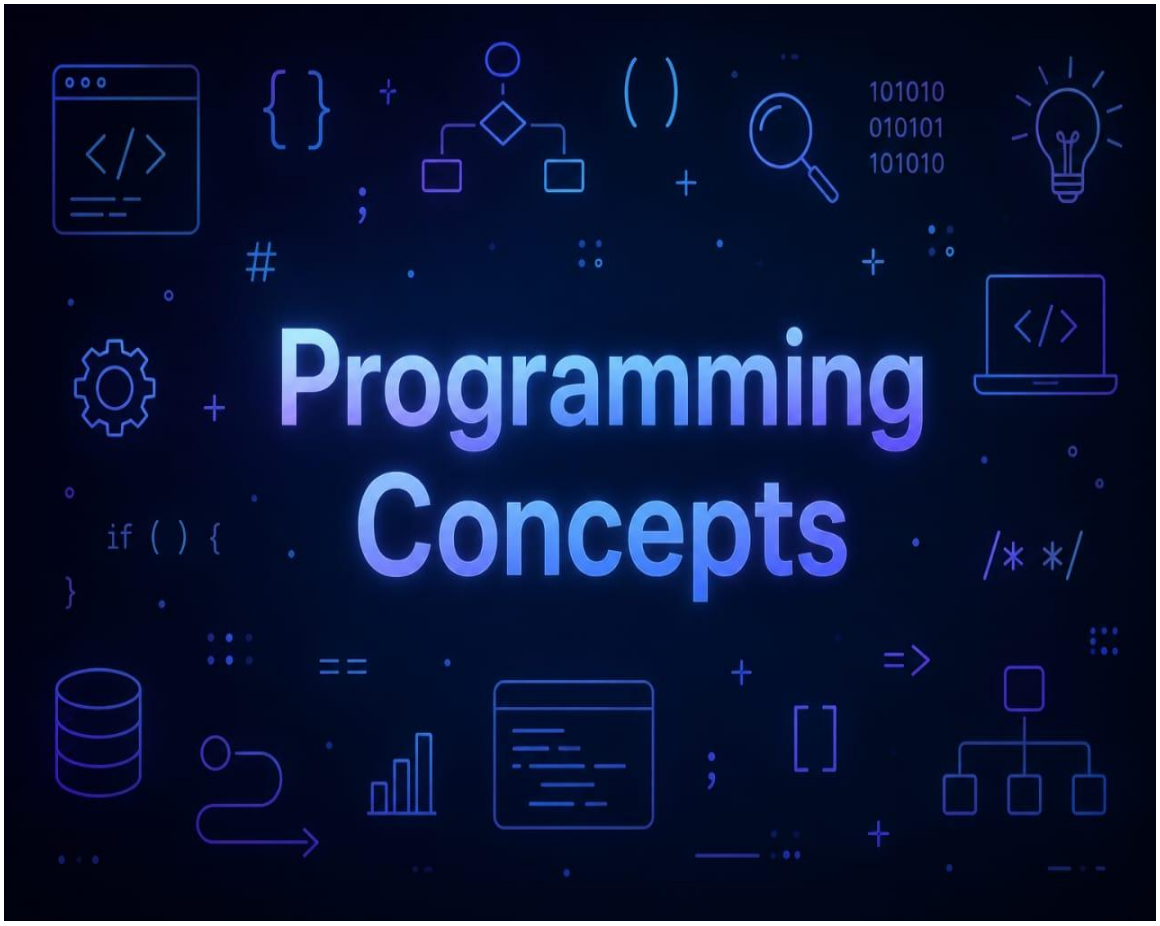


Concepts of Programming Languages

Content:

Chapter 01 : Preliminaries

01*Reasons for Studying Concepts of Programming Languages:

• Increased ability to express ideas:

–Depth at which people think is influenced by the expressive power of the language in which they communicate.

– Language of software development can place limits on the kinds of control structures, data structures, and abstractions they can use.

–The study of programming language concepts builds an appreciation for valuable language features and constructs and encourages programmers to use them, even when the language they are using does not directly support such features and constructs.

•Improved background for choosing appropriate languages:

–Many programmers continue to use the language with which they are most familiar, even if it is poorly suited to new projects.

–If they are familiar with other languages available, they would be in a better position to make informed language choices.

–Features Simulation? It is preferable to use a feature whose design has been integrated into a language than to use a simulation of that feature (less elegant and less safe).

```
for i in range(7):
   print(i)
```

Vs

```
for i in [0,1,2,3,4,5,6]:
   print(i)
```

```
name='bob'
print(f'Welcome {name}!')
```

Vs

```
name='bob'
print('Welcome ' + name + '!')
```

•Increased ability to learn new languages:

–Programming languages are still in a state of continuous evolution, which means continuous learning is essential

–Once a thorough understanding of the fundamental concepts of languages is acquired, it becomes easier to see how concepts are incorporated into the design of the language being learned.

Example 1: OOP might quickly adapt to a new language

Example 2: higher order functions

```
L=[1,2,3,4,5]
Squared=L.map(x=>x*x) // [1, 4, 9, 16, 25]
```

– It is essential that practicing programmers know the vocabulary and fundamental concepts of PL so they can read and understand PL descriptions and evaluations, and promotional literature for languages and compilers.

•Better understanding of significance of implementation:

– Understanding of implementation issues leads to an understanding of why languages are designed the way they are.

– This in turn leads to the ability to use a language more intelligently, as it was designed to be used.

– In some cases, some knowledge of implementation issues provides hints about the relative efficiency of alternative constructs that may be chosen for a program.

```
// Inefficient string concatenation
String result = "";
for (int i = 0; i < 5; i++) {
 result += “Bonjour" + “, ";
}
System.out.println(result);
// Efficient string concatenation using StringBuilder
StringBuilder resultBuilder = new StringBuilder();
for (int i = 0; i < 5; i++) {
 resultBuilder.append(“Bonjour, ");
}
String result = resultBuilder.toString();
System.out.println(result);
```

•Better use of already-known languages:

– Most contemporary programming languages are large and complex.

– It is uncommon for a programmer to be familiar with and use all of the features of a language s/he uses.

– By studying the concepts of programming languages, programmers can learn about previously unknown and unused parts of the languages they already use and begin to use those features.

```
// Traditional Function in JS
function add(a, b){
  return a+b;
    }
// Arrow functions in JS
const add=(a,b)=> a+b;
// Traditional list in python
squars =[]
for i in range(5):
   squares.append(i*i)
// List comprehension in Python
squares=[i*i for i in range(5)]
// Iterating over numbers in Java
List<Integers> li =Arrays.asList(1,2,3,4,5)
for(Integer i: li)
   sout(i)
// Iterating over numbers using lambda
List<Integers> li =  Arrays.asList(1,2,3,4,5)
li.forEach(i->sout(i))
```

•Overall advancement of computing:

–In some cases, a language became widely used, at least in part, because those in positions to choose languages were not sufficiently familiar with P/L concepts.

–Many believe that ALGOL 60 was a better language than Fortran; however, Fortran was most widely used. It is attributed to the fact that the programmers and managers didn’t understand the conceptual design of ALGOL 60.

–In general, if those who choose languages were well informed, perhaps better languages would eventually squeeze out poorer ones.

02*Programming Domains:

•Scientific applications:
–Large numbers of floating point computations; use of arrays
–Fortran

•Business applications
–Produce reports, use decimal numbers and characters
–COBOL

•Artificial intelligence
–Symbols rather than numbers manipulated; use of linked lists
–LISP

•Systems programming
–Need efficiency because of continuous use
–C

•Web software
–Markup (e.g., HTML)
–Scripting (e.g., PHP)

•General-purpose languages:
–e.g. Java

•Special-purpose languages:
–e.g. RPG (Report Program Generator) for business applications

03*Language Evaluation Criteria:

•Readability: The ease with which programs can be read and understood

•Writability: The ease with which a language can be used to create programs

•Reliability: Conformance to specifications (i.e., performs to its specifications)

•Cost: The ultimate total cost Evaluation Criteria: Readability

•Overall simplicity
–A manageable set of features and constructs
–Minimal feature multiplicity and operator overloading

•Orthogonality
– A relatively small set of primitive constructs can be combined in a relatively small number of ways

Assembly 1

```
A reg1, memory_cell
Ar reg1, reg2
```

Assembly 2

```
ADDL operand_1, operand_2
Orthogonal since eiher operand can be a register or a memory cell
```

Syntax considerations:
–Identifier forms: flexible composition
–Special words and methods of forming compound statements
Example:

```
Fortran: do while
 statements
 end do
 if statements
 end if
```

–Form and meaning: self-descriptive constructs, meaningful keywords
–Every possible combination is legal

•Data types
–Adequate predefined data types

•Syntax considerations
–Identifier forms: flexible composition
–Special words and methods of forming compound statements
–Form and meaning: self-descriptive constructs, meaningful keywords

04*Evaluation Criteria: Writability

•Simplicity and Orthogonality
–Few constructs, a small number of primitives, a small set of rules for combining them

•Support for abstraction
–The ability to define and use complex structures or operations in ways that allow details to be ignored

•Expressivity
–A set of relatively convenient ways of specifying operations
–Strength and number of operators and predefined functions

05*Evaluation Criteria: Reliability

•Type checking
–Testing for type errors

•Exception handling
–Intercept run-time errors and take corrective measures

•Aliasing
–Presence of two or more distinct referencing methods for the same memory location

•Readability and Writability
–A language that does not support “natural” ways of expressing an algorithm will require the use of “unnatural” approaches, and hence reduced reliability

06*Evaluation Criteria: Cost

• Training programmers to use the language
• Writing programs (closeness to particular applications)
• Compiling programs
• Executing programs
• Language implementation system: availability of free compilers
• Reliability: poor reliability leads to high costs
• Maintaining programs

07*Language Categories:

•Imperative
–Central features are variables, assignment statements, and iteration
–Include languages that support object-oriented programming
–Include scripting languages
–Include the visual languages
–Examples: C, Java, Perl, JavaScript, Visual BASIC .NET, C++

•Functional
–Main means of making computations is by applying functions to given parameters
–Examples: LISP, Scheme, ML, F#

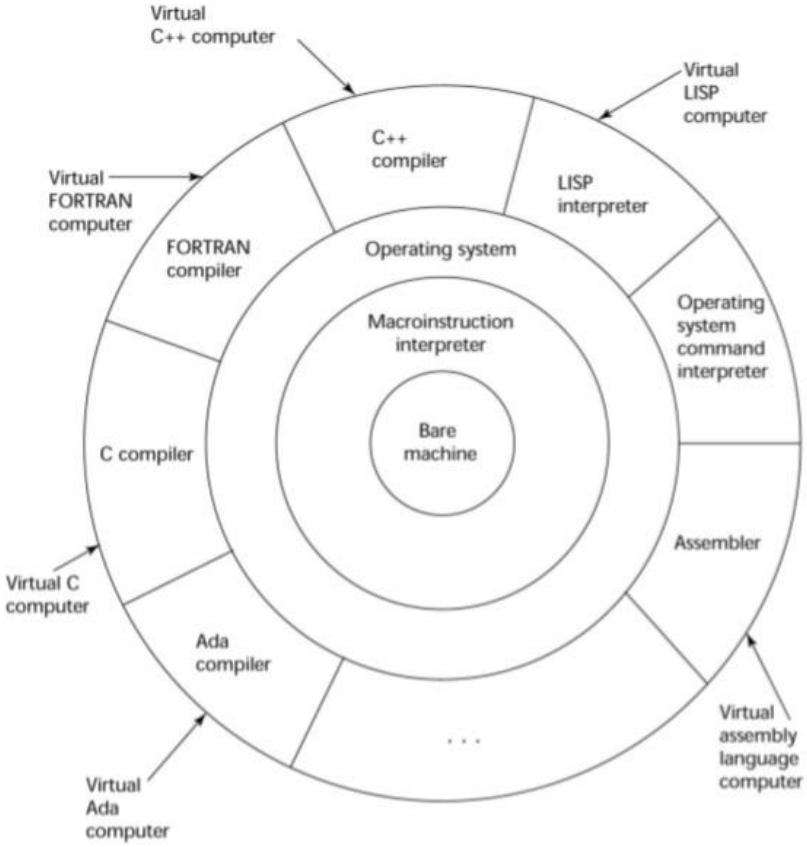
•Logic
–Rule-based (rules are specified in no particular order)
–Example: Prolog

•Markup/Programming Hybrid
–Markup languages extended to support some programming
–Examples: JSTL, XSLT

08*Implementation Methods:

•Compilation
–Programs are translated into machine language; includes JIT systems
–Use: Large commercial applications

•The operating system and language implementation are layered over machine interface of a computer



•Translate high-level program (source language) into machine code.

•Compilation process has several phases:
–lexical analysis: converts characters in the source program into lexical units
–syntax analysis: transforms lexical units into parse trees which represent the syntactic structure of program
–Semantics analysis: generate intermediate code
–code generation: machine code is generated

09*Just-in-Time Implementation Systems:

•Step 1: translate programs into an intermediate language

•Step 2: compile the intermediate language of the subprograms into machine code when they are called
–Note 1: machine code version is kept for subsequent calls
– Note 2: JIT systems are widely used for Java programs
–Note 3: .NET languages are implemented with a JIT system
–Note 4: in essence, JIT systems are delayed compilers

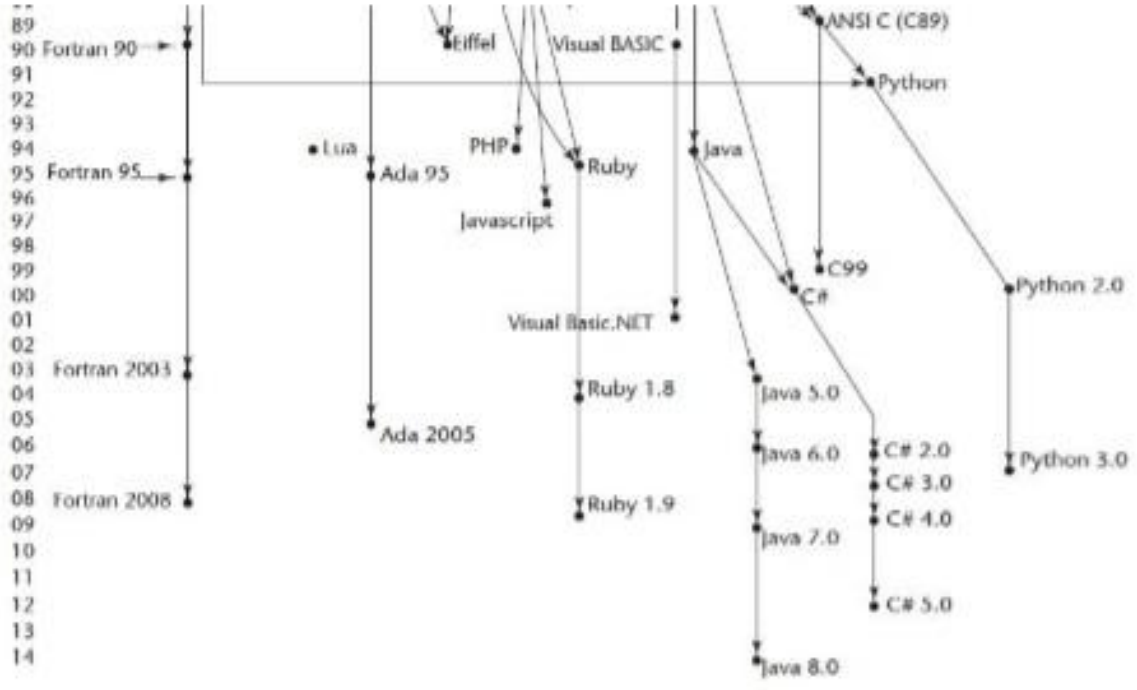
10*Programming Environments:

•UNIX
–An older operating system and tool collection
–Nowadays often used through a GUI (e.g., CDE, KDE, or GNOME) that runs on top of UNIX

•Microsoft Visual Studio.NET
–A large, complex visual environment
–Used to build Web applications and non-Web applications in any .NET language

•NetBeans
–Related to Visual Studio .NET, except for applications in Java

11*Genealogy of common high-level programming languages:



Chapter 02 : Describing Syntax and Semantics

01*Introduction:

•Syntax: the form or structure of the expressions, statements, and program units (Example1: while (boolean_expr) statement)

•Semantics: the meaning of the expressions, statements, and program units
–The semantics of Example1 is: when boolean_expr is true, statement is executed. If boolean_expr is false, control transfers to the statement following the while construct.)

•Syntax and semantics provide a language's definition
–Users of a language definition

•Other language designers

•Implementers

•Programmers (the users of the language)

02*The General Problem of Describing Syntax:

•A metalanguage is a language used to describe another language. (Example: English used to describe Java)

•A sentence is a string of characters over some alphabet

•A language is a set of sentences
–a language is specified by a set of rules

•A lexeme is the lowest level syntactic unit of a language (e.g., *, sum, begin)

•A token is a category of lexemes (e.g., identifier)

Consider the following Java statement:

```
index = 2 * count + 17;
```

Lexeme	Token
index	identifier
=	equal_sign
2	int_literal
*	mult_operator
count	identifier
+	plus_operator
17	int_literal
;	semicolon

•Recognizers
–Read a string and decide whether it follows the rules for the language
–Example: syntax analysis part of a compiler

•Generators
–A device that generates sentences of a language (BNF)
–More useful for specifying the language than for checking a string

•Backus-Naur Form (BNF) and Context-Free Grammars
–Most widely known method for describing programming language syntax
–Developed as part of the process for specifying ALGOL
–Define a class of languages called context-free languages
–BNF is a metalanguage for programming languages.

•Extended BNF (Not included)
–Improves readability and writability of BNF Backus-Naur Form and

03*Context-Free Grammars:

•Context-Free Grammars
–Chomsky described four classes of grammars

Type	Characteristics
0	Unrestricted
1	Context-sensitive
2	Context-free
3	Regular

–Types 2 and 3 are useful for programming language specifications

•Origins of Backus-Naur Form:
–Backus developed notation for programming language specifications syntax (ALGOL58).

•Fundamentals:
–Abstractions are used to describe syntactic structures

•Example: A Java assignment statement might be represented by the abstraction <assign>

•The actual definition of <assign> can be given by <assign> → <var> = <expression>
–Terminals used are Lexemes and Tokens
–Example 1 of BNF rule (or production):

<assign> → <var> = <expression>

example: total = subtotal1 + subtotal2
–Example 2 of BNF rules:

<if_stmt> → if (<logic_expr>) <stmt>
<if_stmt> → if (<logic_expr>) <stmt> else <stmt>
<stmt> → <single_stmt>
| begin <stmt_list> end

–A rule has a left-hand side (LHS) and a right-hand side (RHS), and consists of terminal and nonterminal symbols
–In a context-free grammar, there can only be one symbol on the LHS
–A grammar is a finite nonempty set of rules
–An abstraction (or nonterminal symbol) can have more than one RHS usually separated by the symbol "|" (logical OR)

•Describing Lists:
–A list of identifiers appearing on a data declaration statement.
–Syntactic lists are described using recursion:

<ident_list> → identifier
| identifier, <ident_list>
<ident_list> → <ident>
| <ident>, <ident_list>
<arg_list> → <expr>
| <expr>, <arg_list>
<param_list> → <param>
| param, <param_list>
<stmt_list> → <stmt>
| <stmt>; <stmt_list>
Example:
<digit> -> 0|1|2|3|4|5|6|7|8|9

<integer> -> <digit>
 | <digit> <integer>

–A Grammar for a Small Language:

<program> → **begin** <stmt_list> **end**

<stmt_list> → <stmt>
 | <stmt> ; <stmt_list>
<stmt> → <var> = <expression>
<var> → A | B | C
<expression> → <var> + <var>
 | <var> – <var>
 | <var>

<program> => **begin** <stmt_list> **end**
 => **begin** <stmt> ; <stmt_list> **end**
 => **begin** <var> = <expression> ; <stmt_list> **end**
 => **begin** A = <expression> ; <stmt_list> **end**
 => **begin** A = <var> + <var> ; <stmt_list> **end**
 => **begin** A = B + <var> ; <stmt_list> **end**
 => **begin** A = B + C ; <stmt_list> **end**
 => **begin** A = B +C; <stmt> **end**
 => **begin** A = B + C ; <var> = <expression> **end**
 => **begin** A = B + C ; B = <expression> **end**
 => **begin** A = B + C ; B = <var> **end**
 => **begin** A = B + C ; B = C **end**

–A derivation is a repeated application of rules, starting with the start symbol and ending with a sentence (all terminal symbols)
–Every string of symbols in the derivation is a sentential form
–A sentence is a sentential form that has only terminal symbols
–A leftmost derivation is one in which the leftmost nonterminal in each sentential form is the one that is expanded
–A derivation may be neither leftmost nor rightmost
–A Grammar for a Simple Assignment Statements

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <id> + <expr>
 | <id> * <expr>
 | (<expr>)
 | <id>

–A Derivative of a program in this language: A = B * (A + C)

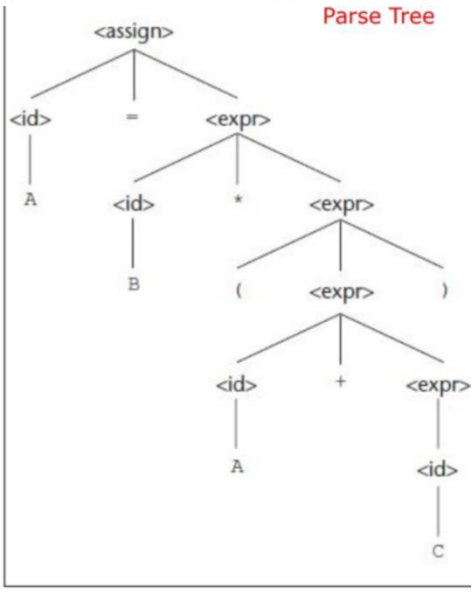
<assign> => <id> = <expr>
=> A = <expr>
=> A = <id> * <expr>
=> A = B * <expr>
=> A = B * (<expr>)
=> A = B * (<id> + <expr>)
=> A = B * (A + <expr>)
=> A = B * (A + <id>)
=> A = B * (A + C)

04*Parse Trees:

–Hierarchical syntactic structures of the sentences of the languages they define.
–An example of a parse tree for the simple statement:
A = B * (A + C)
Note that the derivation below of
this statement was described in the previous lecture:

<assign> => <id> = <expr>
=> A = <expr>
=> A = <id> * <expr>
=> A = B * <expr>
=> A = B * (<expr>)
=> A = B * (<id> + <expr>)
=> A = B * (A + <expr>)
=> A = B * (A + <id>)
=> A = B * (A + C)

Parse Tree

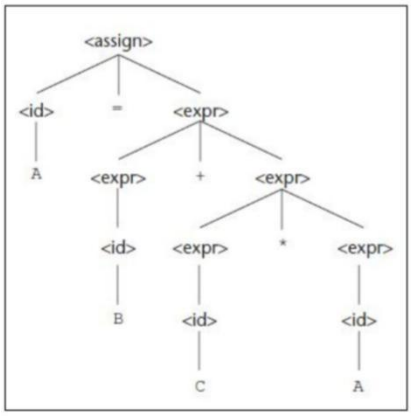


05*Ambiguity:

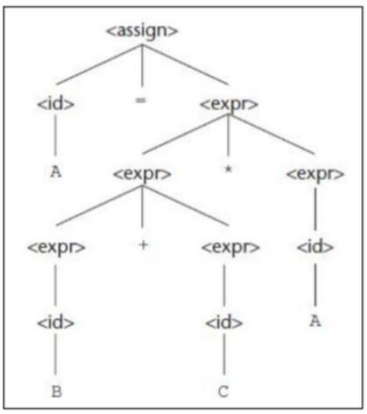
–A grammar is ambiguous if and only if it generates a sentential form that has two or more distinct parse trees
–The grammar of the example below is ambiguous because the sentence A = B + C * A has two distinct parse trees show in the next slide.

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <expr>
| <expr> * <expr>
| (<expr>)
| <id>

–The two distinct parse trees for the same sentence: A = B + C * A



Parse Tree 1



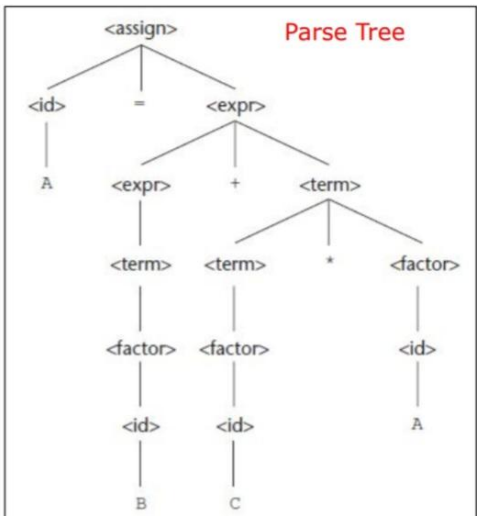
Parse Tree 2

06*Operator Precedence:

–If we use the parse tree to indicate precedence levels of the operators, we cannot have ambiguity
–The grammar of the example below is unambiguous because the sentence A = B + C * A has only one parse tree shown in the next slide.

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <term>
| <term>
<term> → <term> * <factor>
| <factor>
<factor> → (<expr>)
| <id>

–Every derivation with an unambiguous grammar has a unique parse tree.



<assign> => <id> = <expr>
=> A = <expr>
=> A = <expr> + <term>
=> A = <term> + <term>
=> A = <factor> + <term>
=> A = <id> + <term>
=> A = B + <term>
=> A = B + <term> * <factor>
=> A = B + <factor> * <factor>
=> A = B + <id> * <factor>
=> A = B + C * <factor>
=> A = B + C * <id>
=> A = B + C * A

Leftmost Derivation

<assign> => <id> = <expr>
=> <id> = <expr> + <term>
=> <id> = <expr> + <term> * <factor>
=> <id> = <expr> + <term> * <id>
=> <id> = <expr> + <term> * A
=> <id> = <expr> + <factor> * A
=> <id> = <expr> + <id> * A
=> <id> = <expr> + C * A
=> <id> = <term> + C * A
=> <id> = <factor> + C * A
=> <id> = <id> + C * A
=> <id> = B + C * A
=> A = B + C * A

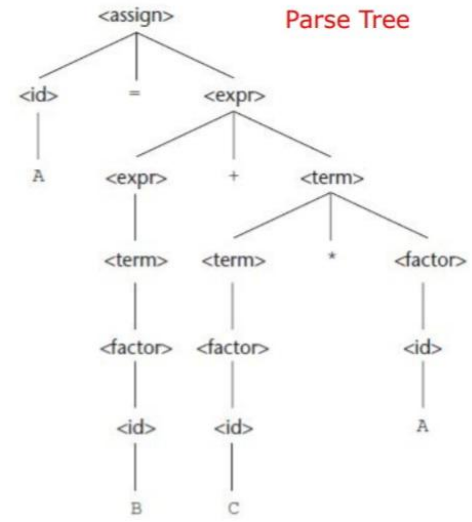
Rightmost Derivation

08*Operator Precedence:

–If we use the parse tree to indicate precedence levels of the operators, we cannot have ambiguity
–The grammar of the example below is unambiguous because the sentence A = B + C * A has only one parse tree shown in the next slide.

```
<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <term>
         | <term>
<term> → <term> * <factor>
         | <factor>
<factor> → ( <expr>)
           | <id>
```

–Every derivation with an unambiguous grammar has a unique parse tree.
–That tree can be represented by different derivations shown in the next slide.



<assign> => <id> = <expr>
=> A = <expr>
=> A = <expr> + <term>
=> A = <term> + <term>
=> A = <factor> + <term>
=> A = <id> + <term>
=> A = B + <term>
=> A = B + <term> * <factor>
=> A = B + <factor> * <factor>
=> A = B + <id> * <factor>
=> A = B + C * <factor>
=> A = B + C * <id>
=> A = B + C * A

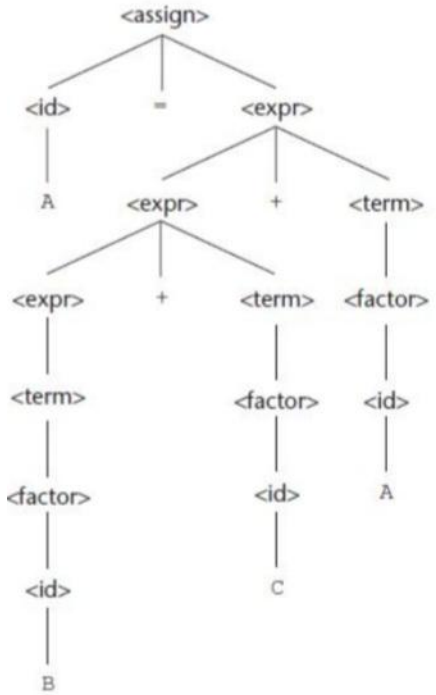
Leftmost Derivation

<assign> => <id> = <expr>
=> <id> = <expr> + <term>
=> <id> = <expr> + <term> * <factor>
=> <id> = <expr> + <term> * <id>
=> <id> = <expr> + <term> * A
=> <id> = <expr> + <factor> * A
=> <id> = <expr> + <id> * A
=> <id> = <expr> + C * A
=> <id> = <term> + C * A
=> <id> = <factor> + C * A
=> <id> = <id> + C * A
=> <id> = B + C * A
=> A = B + C * A

Rightmost Derivation

09*Associativity of Operators:

–When an expression includes two operators that have the same precedence, a semantic rule is required to specify which should have precedence.
This rule is named associativity.
–A parse tree for A = B + C + A using the grammar described in slide 6.



10*An Unambiguous Grammar for if-else:

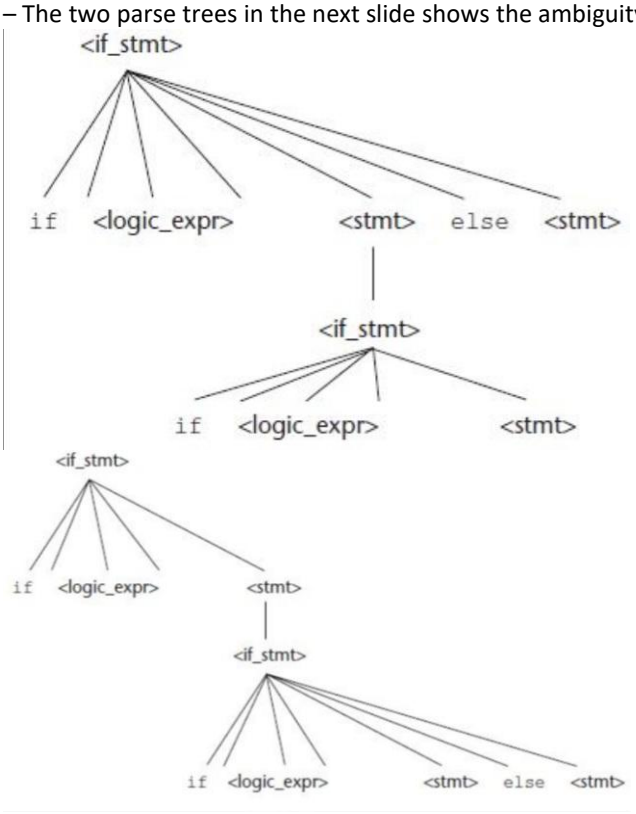
– The BNF rule for a Java if-else statement are as follows:

<if_stmt> → if (<logic_expr>) <stmt>
<if_stmt> → if (<logic_expr>) <stmt> else <stmt>

–If we have <stmt> → <if_stmt> , this grammar is ambiguous. The simplest sentential form that illustrates this ambiguity is

if (<logic_expr>) if (<logic_expr>) <stmt> else <stmt>

– The two parse trees in the next slide shows the ambiguity of this sentential form.



11*Attribute Grammars:

•Static Semantics

–Some syntax rules cannot be specified in BNF such as “all variables must be declared before they are referenced.”

–The static semantics of a language is only indirectly related to the meaning of programs during execution.

–Static semantics is so named because the analysis required to check these specifications can be done at compile time.

–Attribute grammars are a formal approach to describe and check the correctness of the static semantics rules of a program.

•Attribute Grammars Defined

–An attribute grammar consists of the following additional features:

•Associated with each grammar symbol X is a set of attributes A(X).

•Associated with each grammar rule is a set of semantic functions f(A(X)) and a possibly empty set of predicate functions over the attributes of the symbols in the grammar rule.

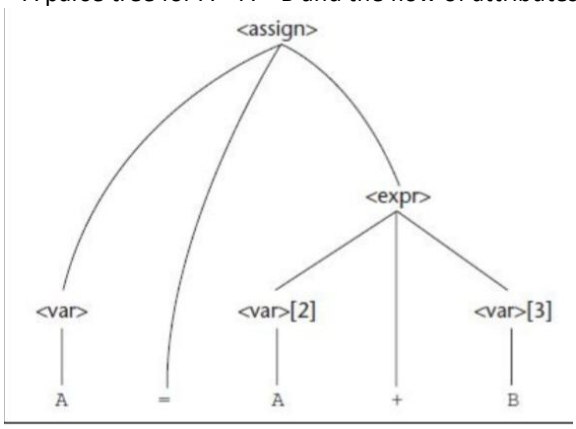
•A predicate function has the form of a Boolean expression on the union of the attribute set.

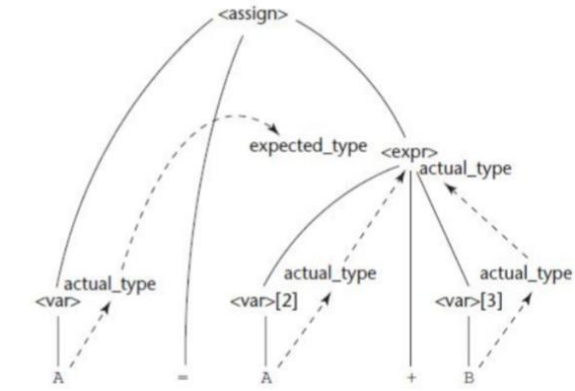
•Examples of Attribute Grammars:

1. Syntax rule: <assign> → <var> = <expr>
   Semantic rule: <expr>.expected_type ← <var>.actual_type
2. Syntax rule: <expr> → <var>[2] + <var>[3]
   Semantic rule: <expr>.actual_type ←
   if (<var>[2].actual_type = int) and
   (<var>[3].actual_type = int)
   then int
   else real
   end if
   Predicate: <expr>.actual_type == <expr>.expected_type
3. Syntax rule: <expr> → <var>
   Semantic rule: <expr>.actual_type ← <var>.actual_type
   Predicate: <expr>.actual_type == <expr>.expected_type
4. Syntax rule: <var> → A | B | C
   Semantic rule: <var>.actual_type ← look-up (<var>.string)

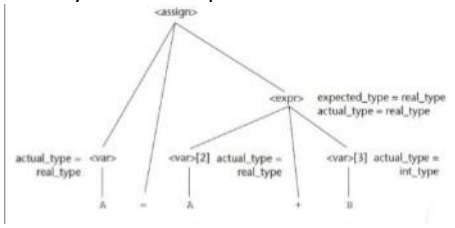
•Computing Attribute Values:

– A parse tree for A = A + B and the flow of attributes in the tree





–A fully attributed parse tree for A = A + B



12*Describing the Meanings of Programs:

Dynamic Semantics:

•There is no single widely acceptable notation or formalism for describing semantics
•Several needs for a methodology and notation for semantics:
–Programmers need to know what statements mean before usage
–Compiler writers must know exactly what language constructs do
–Correctness proofs would be possible
–Compiler generators would be possible
–Designers could detect ambiguities and inconsistencies

•Operational Semantics:

–Describe the meaning of a program by executing its statements on a machine, either simulated or actual.The change in the state of the machine (memory, registers, etc.) defines the meaning of the statement
–To use operational semantics for a high-level language, a virtual machine is needed
–A hardware pure interpreter would be too expensive
–A software pure interpreter also has two problems
•The detailed characteristics of the particular computer would make actions difficult to understand
•such a semantic definition would be machine- dependent
–A better alternative: A complete computer simulation
•Process (step 1): Build a translator (translates source code to the machine code of an idealized computer)
•Process (step 2): Build a simulator for the idealized computer

Evaluation of operational semantics:

•Good if used informally (language manuals, etc.)
•Extremely complex if used formally (e.g., VDL), it was used for describing semantics of PL/I.

The Basic Process:

•Uses of operational semantics:
–Language manuals and textbooks
–Teaching programming languages
•Two different levels of uses of operational semantics:
–Natural operational semantics
–Structural operational semantics

•Example: Semantic of the C for construct can described in terms of simpler statements, as in

C Statement	*Meaning*
`for (expr1; expr2; expr3) {`	`expr1;`
`...`	`loop: if expr2 == 0 goto out`
`}`	`...`
	`expr3;`
	`goto loop`
	`out: ...`

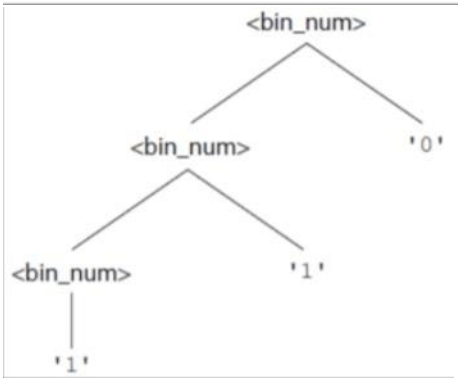
•Denotational Semantics:

–Based on recursive function theory
–Most abstract semantics description method
–The process of building a denotational specification for a language:
•Define a mathematical object for each language entity
•Define a function that maps instances of corresponding mathematical objects
–The meaning of language constructs are defined by only the values of the program's variables.

– Example:

• Syntax of binary numbers and its corresponding parse tree (value 110)

```
<bin_num> → '0'
          | '1'
          | <bin_num> '0'
          | <bin_num> '1'
```

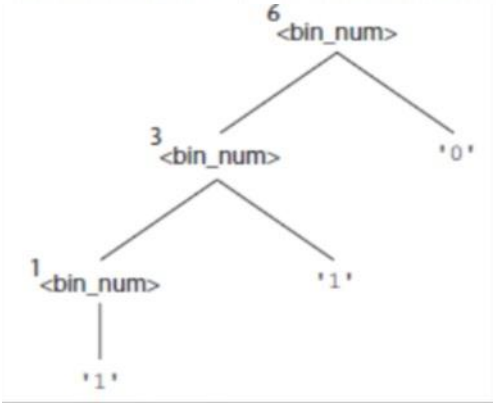


Describing the Meanings of Programs:

–Example1 :

•Denotation Mapping Mbin for the binary sequence '110'

M_{bin}('0') = 0
M_{bin}('1') = 1
M_{bin}(<bin_num> '0') = 2 * M_{bin}(<bin_num>)
M_{bin}(<bin_num> '1') = 2 * M_{bin}(<bin_num>) + 1



–Example 2:

•Syntax of decimal numbers and its corresponding denotational mapping

```
<dec_num> → '0'|'1'|'2'|'3'|'4'|'5'|'6'|'7''8'|'9'
          |<dec_num> ('0'|'1'|'2'|'3'|'4'|'5'|'6'|'7'|'8'|'9')
```

M_{dec}('0') = 0, M_{dec}('1') = 1, M_{dec}('2') = 2, . . ., M_{dec}('9') = 9
M_{dec}(<dec_num> '0') = 10 * M_{dec}(<dec_num>)
M_{dec}(<dec_num> '1') = 10 * M_{dec}(<dec_num>) + 1
. . .
M_{dec}(<dec_num> '9') = 10 * M_{dec}(<dec_num>) + 9

–Expressions:

•BNF Description of Expressions in Most Programming Languages

Grammar

<expr> → <dec_num> | <var> | <binary_expr>
<binary_expr> → <left_expr> <operator> <right_expr>
<left_expr> → <dec_num> | <var>
<right_expr> → <dec_num> | <var>
<operator> → + | *

13*Describing the Meanings of Programs: Dynamic Semantics

•Axiomatic Semantics:

–Based on formal logic (predicate calculus)

–Original purpose: formal program verification

–Axioms or inference rules are defined for each statement type in the language (to allow transformations of logic expressions into more formal logic expressions)

–The logic expressions are called assertions

–Assertions:

•An assertion before a statement (a precondition) states the relationships and constraints among variables that are true at that point in execution

•An assertion following a statement is a postcondition

•A weakest precondition is the least restrictive precondition that will guarantee the postcondition

•Pre (P) and Post (Q) conditions are in the form: {P} statement(s) {Q}

•Example: {b > 10} a = b + 1 {a > 1}

•Answer:

1)Postcondition: a > 1

2)Statement: a = b + 1

3)Possible Precondition: b > 10 ?

4)Weakest Precondition: b > 0 ?

–Weakest Preconditions:

•Least restrictive precondition that will guarantee the validity of the associated postcondition

•Example: sum = 2 * x + 1 {sum > 1}

–Possible preconditions: {x > 10}, {x > 50}, {x > 100}

–Weakest precondition: {x > 0}

•Weakest precondition can be computed from the most general postcondition

•An inference rule is a method of inferring the truth of one assertion on the basis of the values of other assertions.

•rule states that (S1, S2, .., and Sn)/S are true, then the truth of S can be inferred.

•An axiom is a logical statement that is assumed to be true. Therefore, an axiom is an inference rule without an antecedent.

–Assignment Statements:

•To define the meaning of an assignment statement there must be a way to compute its precondition from its postcondition.

•Let x = E be a general assignment statement and Q be its postcondition. Then, its weakest precondition, P, is defined by the axiom P = Qx --> E which means that P is computed as Q with all instances of x replaced by E.

•Example 1: a = b / 2 – 1 {a < 10}
–Compute weakest precondition by replacing the postcondition as following: b / 2 – 1 < 10
–As a result, weakest precondition is {b < 22}

•Example 2: x = 2 * y – 3 {x > 25}
–Compute weakest precondition: 2 * y - 3 > 25
– As a result, weakest precondition is {y > 14}

• Example 3: x = x + y – 3 {x > 10}
– Compute weakest precondition: x + y – 3 > 10
– As a result, weakest precondition is {y > 13 - x}

• Example 4: {x > 3} x = x – 3 {x > 0}
– Compute weakest precondition: {x > 3}
– The example logic statement is proven

•Example 5: {x > 5} x = x – 3 {x > 0}
–Compute weakest precondition: {x > 3}
–Using the rule of consequence {x > 5} {x > 3}
–The consequence is true

–Sequences:
•The weakest precondition for a sequence of statements cannot be described by an axiom, because the precondition depends on the particular kinds of statements in the sequence.
•Using inference rule, let S1 and S2 be adjacent program statements. If S1 and S2 have the following pre- and post-conditions

$$\{P1\}\ S1\ \{P2\} \text{ and } \{P2\}\ S2\ \{P3\}$$

The inference rule for such a two-statement sequence is

$$\frac{\{P1\}S1\ \{P2\}, \{P2\}\ S2\ \{P3\}}{\{P1\}S1, S2\ \{P3\}}$$

Chapter 03 : Names , Bindings , and Scopes:

01*Introduction:

Imperative languages are abstractions of von Neumann architecture:
–Memory (storing instructions & data)
–Processor (operations for modifying contents of the memory)
–Characteristics of abstraction similar to characteristics of cells;
ex: an integer variable is usually represented directly in one or more bytes of memory
–A variable can be characterized by a collection of properties, or attributes (type - designing a data typ) Note that the phrase C-based languages in the book refers to C, Objective-C, C++, Java, and C#.

02*Names:

•Design Issues for Names/Identifiers:
–used with variables, subprograms, parameters, other constructs
–Maximum length?
–Are names case sensitive?
–Are special words reserved words or keywords?

Length:
–Name should be connotative

• Example: to declare “length”– use the word instead of “L”
– Name wants reasonable length represent for symbol table

•FORTRAN I: maximum 6
•COBOL: maximum 30
•FORTRAN 90 and ANSI C: maximum 31
•Ada and Java: no limit, and all are significant
•C++: no limit, but implementers often impose one

•Case Sensitivity:
–Disadvantage: readability
•Names that look alike are different

•Worse in C++ and Java because predefined names are mixed case (e.g. IndexOutofBoundsException)
–C, C++, and Java names are case sensitive
–The names in many other languages are not
–“Camel” notation – myItem – embedded uppercase – is a style choice, not a language decision
Special Words:
–An aid to readability; used to delimit or separate statement clauses such as „;‟ – “{ }‟ – etc.
–A keyword is a word that is special only in certain contexts, e.g. in Fortran:

•Real VarName (Real is a data type followed with a name, therefor Real is a keyword)
•Real = 3.4 (Real is a variable)
–A reserved word is a special word that cannot be used as a user-defined name.

03*Variable:

Variable Name:

–A program variable is an abstraction of a computer memory cell or collection of cells.
–A variable can be characterized as a sextuple of attributes: name, address, value, type, lifetime, and scope
–Discussion of variable attributes will lead to examinations of the important related concepts of aliases, binding, binding times, declarations, scoping rules, and referencing environments.
– Most variables have names, which can be sometimes called its l-value.

Variable Address:

–Address of a variable is the machine memory address with which it is associated.
–In some languages, it is possible for the same variable to be associated with different addresses at different times during the program execution.
•For example, if a subprogram has a local variable that is allocated from the run-time stack when the subprogram is called, different calls may result in that variable having different addresses.
–It is possible to have multiple variables that have the same address; variables are called aliases.
–Aliases are created via pointers, reference variables, C and C++ unions (discussed more in Chapter 6).
–Aliasing can be created in many languages through subprogram parameters (discussed in Chapter 9).
–Aliases are harmful to readability (program readers must remember all of them).

Variable Type:
–The type of a variable determines the range of values the variable can store and the set of operations that are defined for values of the type.
•For example, the int type in Java specifies a value range of
−2147483648 to 2147483647 and arithmetic operations for addition, subtraction, multiplication, division, and modulus.
•Variable Value
–The value of a variable is the contents of the memory cell or cells associated with the variable.
–A variable‟s value is sometimes called its r-value because it is what is required when the name of the variable appears in the right side of an assignment statement.
•To access the r-value, the l-value must be determined first.

04*The Concept of Binding:

•A binding is an association, such as between:
–An attribute and an entity (e.g., type of a variable),
–An operation and a symbol (e.g., meaning of *),
–A function and its definition

•Binding time is the time at which a binding takes place:
–Design time: bind operator symbols (e.g., *) to operations
–Implementation time: bind floating point type to a representation
–Compile time: bind a variable to a type
–Link time: bind library subprogram to code
–Load time: bind a C static variable to a memory cell
–Run time: bind a non-static local variable to a memory cell

Example: Consider the following C++ assignment statement:

```
count = count + 5;
```

–The type of count is bound at compile time.
–The set of possible values of count is bound at compile time.
–The meaning of the operator symbol + is bound at compile time, when the types of its operands have been determined.
–The internal representation of the literal 5 is bound at design time.
–The value of count is bound at execution time.

•Binding of Attributes to Variables:
–A binding is static if it first occurs before run time and remains unchanged throughout program execution.
–A binding is dynamic if it first occurs during execution or can change during execution of the program

•Binding Questions
–How is a type specified?
–When does the binding take place?

Type Bindings:
– Static Type Binding:
• If static, the type may be specified by either an explicit or an implicit declaration
• An explicit declaration is a program statement used for declaring the types of variables: int count;
• An implicit declaration is a default mechanism for specifying types of variables (the first appearance of the variable in the program)
• FORTRAN, PL/I, BASIC, and Perl provide implicit declarations
– Advantage: writability
– Disadvantage: reliability

•Example: Perl Type Binding
–Begin with $ is scalar (string or numeric): $a = 10;
–Begin with @ is array: @ages = (25, 30, 40);
–Begin with % is hash: %data = ('John Paul', 45, 'Lisa', 30, 'Kumar', 40);
–Different namespaces
–Advantage: can tell from name what type of data it is

•Type not specified by declaration, not determined by name (JavaScript and PHP)
•Specified through an assignment statement e.g., JavaScript

```
list = [2, 4.33, 6, 8];
list = 17.3;
```

Advantage: flexibility (generic program units)
Disadvantages:
–High cost (dynamic type checking requires run-time descriptors)
–Type error detection by the compiler is difficult

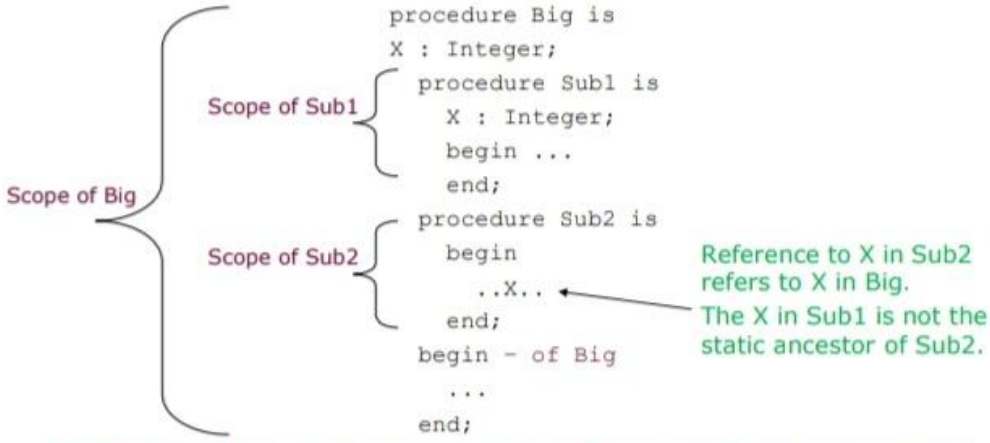
•Example of dynamic type

```
i = x; // desired
i = y; // typed accidentally, y is array
```

Later uses of i expect a scalar, but it is now an array so incorrect results occur.
With static binding, i = y; would generate an error, easier to find problem.

•The scope of a variable is the range of statements over whichit is visible.
•A variable is visible in a statement if it can be referenced orassigned in that statement.
•The scope rules of a language determine how references tonames are associated with variables.
•variable is local in a program unit or block if it is declaredthere.
•The nonlocal variables of a program unit are those that are visible but not declared there such as global variables.

•Static Scope:
–Based on program text
–To connect a name reference to a variable, you (or the compiler) must find the declaration
–Search process: search declarations, first locally, then inincreasingly larger enclosing scopes, until one is found for the given name
–Enclosing static scopes (to a specific scope) are called itsstatic ancestors; the nearest static ancestor is called a static parent.



C & C++ do not allow nested procedures. Nor does Java. Inner classes may accomplish some of same goals

\

05*Blocks:

–The C-based languages allowany compound statement (a statement sequence surrounded by matched braces) to have declarations and thereby define a new scope.
–Such compound statements are called blocks.

```
if(list[i] < list[j])
{
    int temp;
    temp = list[i];
    list[i] = list[j];
    list[j] = temp;
}
```

–Variables can be hidden from a unit by having a "closer" variable with the same name
–illegal in Java and C#, legal in C and C++
–C++ and Ada allow access to these "hidden" variables
•In Ada: unit.name
•In C++: class_name::name

```
void sub() {
    int count;
    while (...) {
        int count;
        count++;
        ...
    }
    ...
}
```

•Declaration Order:
–In C#, the scope of anyvariable declared in ablock / method is thewhole block, regardless of the position of the declaration in the block,as long as it is not in a nested block.

```
{int x;
 ...
   {int x; //illegal
    ...
   }
 ...
}
```

```
{...
   {int x; //illegal
    ...
   }
 int x;
}
```

06*Global Scope:

–Definitions outside functions in a file create global variables,which potentially can be visible to those functions.
–Declarations specify types and other attributes but do not causeallocation of storage.
–A global variable in C is implicitly visible in all subsequentfunctions in the file, except those that include a declaration of alocal variable with the same name.

```
--- Javascript ---
var carName = "Volvo";//globally
// code here can use carName
function myFunction() {
  // code here can use carName
}
```

```
--- C ---
int a = 1, b = 5; //globally
void main() {
  int b = 7; //locally
  // code here use global a
  // code here cannot use global b
}
```

07*Dynamic Scope:

–Based on calling sequences of program units, not their textual layout (temporal versus spatial)
–References to variables are connected to declarations by searching back through the chain of subprogram calls. It must be determined at runtime
–The scope of variables in APL, SNOBOL4, and the early versionsof Lisp is dynamic. Perl and Common Lisp also allow variables to be declared to have dynamic scope, although the default scoping mechanism in these languages is static.
–Example 1: big calls sub1 calls sub2
•The search proceeds from the local procedure, sub2, to its caller, sub1,where a declaration for x is found. x value is 7
– Example 2: big calls sub2 directly
•The dynamic parent of sub2 is big,and the reference is to the x
declared in big. x value is 3

```
function big()
{
   function sub1()
   {
      var x = 7;
   }

   function sub2()
   {
      var y = x;
      var z = 3;
   }
   var x = 3;
}
```

08*Scope and Lifetime:

•The scope and lifetime of a variable appear to be related.
•For example, consider a variable that is declared in a Javamethod that contains no method calls.
–The scope of such a variable is from its declaration to the end of the method.
–The lifetime of that variable is the period of time beginning when the method is entered and ending when execution of the method terminates.
•Scope and lifetime are also unrelated when subprogram calls are involved.
•Consider the following C++ functions:
–The scope of the variable sum is completely contained within the function compute only.
–The lifetime of sum extends over the time during which printheader executes.
–Whatever storage location sum is bound to before the call to printheader,that binding will continue during and after the execution of printheader.

```
void printheader() {
    . . .
} // end method

void compute() {
    int sum;
    . . .
    printheader();
} // end method
```

09*Referencing Environments:

•The referencing environment of a statement is the collectionof all variables that are visible in the statement.
•In a static-scoped language, it is the local variables plus all the visible variables of its ancestors.
•In a dynamic-scoped language, it is the local variables plus all visible variables in all active subprograms.
–A subprogram is active if its execution has begun but has not yet terminated.
•In Python, scopes can be created by function definitions (new environment for each scope created)
• Consider the following Python skeletal program:

```
g = 3; # A global

def sub1():
    a = 5; # Creates a local
    b = 7; # Creates another local
    . . . <------------------------------ 1
  def  sub2():
  global g;  # Global g is now assignable here
    c = 9;  # Creates a new local
    . . . <------------------------------ 2
    def  sub3():
      nonlocal  c:  # Makes nonlocal c visible here
      g = 11;  # Creates a new local
      . . . <------------------------------ 3
```

• The referencing environments of the indicated program points are as follows:

Point	*Referencing Environment*
1	local `a` and `b` (of `sub1`), global `g` for reference, but not for assignment
2	local `c` (of `sub2`), global `g` for both reference and for assignment

10*Dynamically scoped language:

•Consider the following program. Assume that the only function calls are the following: main calls sub2, which calls sub1

```
void sub1() {
  int a, b;
  . . . <------------ 1
}  /* end of sub1 */
void sub2() {
  int b, c;
  . . . . <------------ 2
  sub1();
}  /* end of sub2 */
void main() {
  int c, d;
  . . . <------------ 3
  sub2();
}  /* end of main */
```

Dynamically scoped language

The referencing environments of the indicated program points are as follows:

Named Constants:

•A named constant is a variable that is bound to a value only once.
•Named constants are useful as aids to readability and program reliability.
•Readability can be improved, for example, by using the name PI instead of the constant 3.14159265.
•Another important use of named constants is to parameterize a program.

Consider the following skeletal Java program segment:

```
void example() {
  int[] intList = new int[100];
  String[] strList =  new  String[100];
  . . .
  for  (index = 0; index < 100; index++) {
    . . .
  }
  . . .
  for  (index = 0; index < 100; index++) {
    . . .
  }
  . . .
  average = sum / 100;
  . . .
}
```

```
void example() {
  final int  len = 100;
  int[] intList =  new int[len];
  String[] strList =  new  String[len];
  . . .
  for (index = 0; index < len; index++) {
    . . .
  }
  . . .
  for (index = 0; index < len; index++) {
    . . .
  }
  . . .
  average = sum / len;
  . . .
}
```

Named Constants:

•Dynamic binding of values to named constants for other languages:
–FORTRAN 90: real, parameter :: pi = 3.1415927;
–Ada: Byte_Per_Page : constant := 512;
–C++: const int result = 2 * width + 1;
–Java: final double PI = 3.1415;
–C#: const (static) and readonly (dynamic)
–Python: Const_Name = “Python”;

Variable Initialization

•The binding of a variable to a value at the time it is bound to storage is called initialization
•Initialization is often done on the declaration statement,
for example in Java:

```
int sum = 0;
String message = “Java”;
char letter = ‘u’;
```

Chapter 04 : Data Types

01*Introduction:

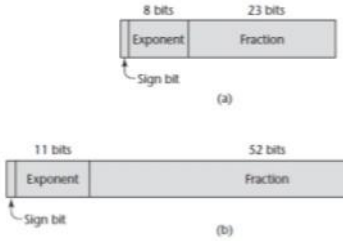
•A data type defines a collection of data objects and a set of predefined operations on those objects
•A descriptor is the collection of the attributes of a variable (an area of memory that stores the attributes of a variable)
•An object represents an instance of a user-defined (abstract data) type
•One design issue for all data types: What operations are defined and how are they specified?

02*Primitive Data Types:

•Almost all programming languages provide a set of primitive data types
•Primitive data types: Those not defined in terms of other data types
•Some primitive data types are merely reflections of the hardware
• Others require only a little non-hardware support for their implementation

•Numeric Types:
–Integer:
•Almost always an exact reflection of the hardware so the mapping is trivial
•There may be as many as eight different integer types in a language
•Java's signed integer sizes: byte, short, int, long
•C++ and C# include unsigned integer types

–Floating-Point:
•Model real numbers, but only as approximations
•Languages for scientific use support at least two floating-point types (e.g., float and double)



–Complex:
•Some languages support a complex type, e.g., C99, Fortran, and Python
•Each value consists of two floats, the real part and the imaginary part
•Literal form (in Python): (7 + 3j), where 7 is the real part and 3 is the imaginary part

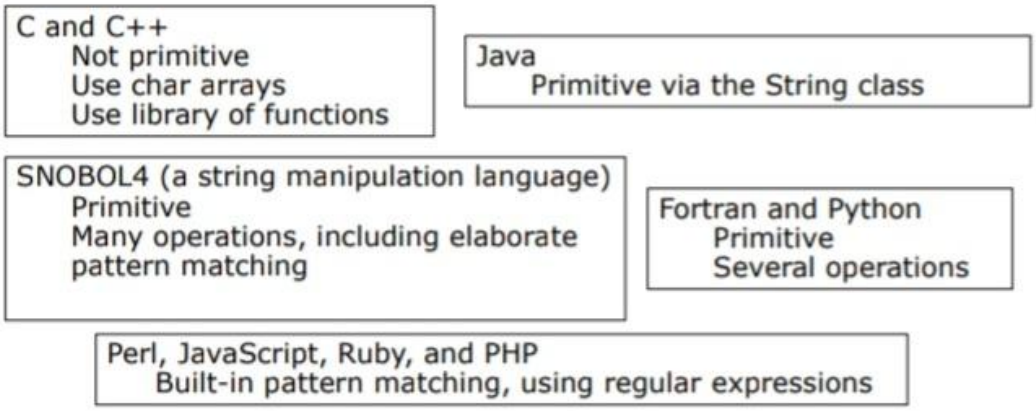
–Decimal:
•For business applications (money)
•Essential to COBOL
•C# offers a decimal data type
•Store a fixed number of decimal digits, in coded form (BCD)
•Advantage: accuracy
•Disadvantages: limited range, wastes memory

•Boolean Types:
–Simplest of all
–Range of values: two elements, one for “true” and one for “false”
–Could be implemented as bits, but often as bytes
–Advantage: readability

•Character Types:
–Stored as numeric coding mostly as ASCII
–An alternative, 16-bit coding: Unicode (UCS-2)
•Includes characters from most natural languages
•Originally used in Java
•C# and JavaScript also support Unicode
–32-bit Unicode (UCS-4)
•Supported by Fortran, starting with 2003

Character String Types:
•Values are sequences of characters
•Design issues:
–Is it a primitive type or just a special kind of array?
–Should the length of strings be static or dynamic?

Character String Types:
•Strings and Their Operations
–Typical operations:
•Assignment and copying
•Comparison (=, >, etc.)
•Catenation
•Substring reference
•Pattern matching
– Character String Type in Certain Languages:



•String Length Options:
–Static: COBOL, Java's String class
–Limited Dynamic Length: C and C++

•In these languages, a special character is used to indicate the end of a string's characters, rather than maintaining the length
–Dynamic (no maximum): SNOBOL4, Perl, JavaScript
–Ada supports all three string length options

•Implementation of Character String Types:
–Static length: compile-time descriptor
–Limited dynamic length: may need a run-time descriptor for length (but not in C and C++)
–Dynamic length:
•Need run-time descriptor;
•Allocation/De-allocation is the biggest implementation problem

03*Enumeration Types:

•Designs:
–All possible values, which are named constants, are provided in the definition. C# example
enum days {mon, tue, wed, thu, fri, sat, sun};
–In languages that do not have enumeration types, programmers usually simulate them with integer values. If C program does not support enumeration:

```
int red = 0, blue = 1, green = 2;
```

•The problem with this approach is that because we have not defined a type for the colors (no type checking)
–C and Pascal were the first widely used languages to include an enumeration data type. C++ includes C's enumeration types. In C++, we could have the following: enum colors {red, blue, green, yellow, black} colors myColor = blue, yourColor = red;
•The colors type uses the default internal values for the enumeration constants: 0, 1, . . . ,
–For example, if the current value of myColor is blue, then the expression myColor++ would assign green to myColor.

04*Array Types:

•An array is an aggregate of homogeneous data elements in which an individual element is identified by its position in the aggregate, relative to the first element.
•In some languages, such as C, C++, Java, and C#, all of the elements of an array are required to be of the same type. Pointers and references are restricted to point to or
reference a single type.
•In other languages, such as JavaScript, Python, and Ruby, variables are typeless references to objects or data values but the arrays still consist of elements of a single type.

Arrays and Indices:

–Indexing (or subscripting) is a mapping from indices to elements array_name (index_value_list) an element
–Index Syntax:
•FORTRAN, PL/I, Ada use parentheses
•Ada explicitly uses parentheses to show uniformity between array references and function calls because both are mappings ex: Sum := Sum + B(I);
•Most other languages such as Java use brackets [ ]
–FORTRAN, C: integer only
–Ada: integer or enumeration (includes Boolean and char)
–Java: integer types only
–Index range checking
•C, C++, Perl, and Fortran do not specify range checking
•Java, ML, C# specify range checking
•In Ada, the default is to require range checking, but it can be turned off

•Subscript Bindings and Array Categories:

–Static: subscript ranges are statically bound and storage allocation is static (before run-time)

•Advantage: efficiency (no dynamic allocation)
–Fixed stack-dynamic: subscript ranges are statically bound, but the allocation is done at declaration time

•Advantage: space efficiency
–Stack-dynamic: subscript ranges are dynamically bound and the storage allocation is dynamic (done at run-time)

•Advantage: flexibility (size of an array known when used)
–Fixed heap-dynamic (similar to fixed stack-dynamic): storage binding is dynamic but fixed after allocation (i.e., binding is done when requested and storage is allocated from heap, not stack)
– Heap-dynamic: binding of subscript ranges and storage allocation is dynamic and can change any number of times

•Advantage: flexibility (arrays can grow or shrink during program execution)

•Subscript Bindings and Array Categories:
–C and C++ arrays that include static modifier are static
–C and C++ arrays without static modifier are fixed stack-dynamic
–C and C++ provide fixed heap-dynamic arrays
–C# includes a second array class ArrayList that provides fixed heap-dynamic
–Perl, JavaScript, Python, and Ruby support heap-dynamic arrays

•Array initialization:
–Some language (C, C++, Java, C#) allow initialization at the time of storage allocation
•C# example: int list [] = {4, 5, 7, 83};
•C & C++ Character strings: char name [] = “freddie”;
•C & C++ Array of strings: char *names [] = {“Bob”, “Jake”, “Darcie”};
•Java String objects: String[] names = {“Bob”, “Jake”, “Darcie”};

–Some other languages:
•Ada: List : array (1..5) of Integer := (1 => 17, 3 => 34, others => 0);
•Python (List comprehensions) list = [x ** 2 for x in range(12) if x % 3 == 0] puts [0, 9, 36, 81] in list

Array Operations:

–APL provides the most powerful array processing operations for vectors and matrixes as well as unary operators (for example, to reverse column elements)

–Ada allows array assignment but also catenation

–Python allows array assignments (but they are only reference changes) and supports array catenation and element membership operations

–Ruby provides array catenation

–Fortran provides elemental operations because they are between pairs of array elements

•For example, + operator between two arrays results in an array of the sums of the element pairs of the two arrays

Associative Arrays:

•Structure and Operations

–An associative array is an unordered collection of data elements that are indexed by an equal number of values called keys

–In Perl, they are called hashes, because in the implementation their elements are stored and retrieved with hash functions.

```
%salaries = ("Gary" => 75000, "Perry" => 57000,
"Mary" => 55750, "Cedric" => 47850);
$salaries{"Perry"} = 58750;
delete $salaries{"Gary"};
@salaries = ();
```

05*Record Types:

•Definitions of Records:

–The record elements or fields are named (not indexed) with identifiers, and references to the fields are made using these identifiers.

–Example of COBOL form of a record declaration:

```
01 EMPLOYEE-RECORD.
   02 EMPLOYEE-NAME.
      05 FIRST          PICTURE IS X(20).
      05 Middle         PICTURE IS X(10).
      05 LAST           PICTURE IS X(20).
  02 HOURLY-RATE       PICTURE IS 99V99.
```

–In Java and C#, records can be defined as data classes, with nested records defined as nested classes. Data members of such classes serve as the record fields.

```
employee.name = "Freddie"
employee.hourlyRate = 13.20
```

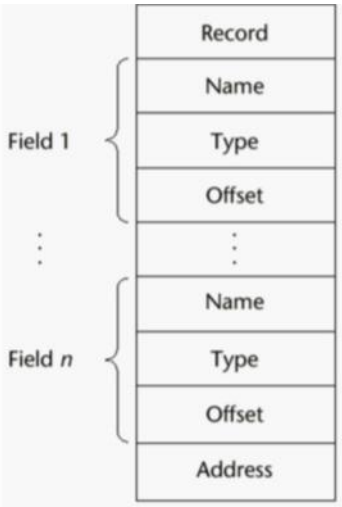
These assignment statements create a table (record) named employee with two elements (fields) named ‘name’ and ‘hourlyRate’, both initialized.

•References to Record Fields
–COBOL : field_name OF record_name_1 OF ... OF record_name_n
Example from slide 3:Middle OF EMPLOYEE- NAME OF EMPLOYEE- RECORD
–Others (dot notation)
record_name_1.record_name_2..... record_name_n.field_name
Example from slide 3:Employee_Record.Employee_Name.Middle

•Fully qualified references must include all record names

•Elliptical references allow leaving out record names as long as the reference is unambiguous, for example in COBOL FIRST, FIRST OF EMP-NAME, and FIRST of EMP-REC are elliptical references to the employee's first name

Implementation of Record Types Offset address relative to the beginning of the records is associated with each field



The compile- time descriptor for a record

06*Tuple Types:

•A tuple is a data type that is similar to a record, except that the elements are not named.
•Python includes an immutable tuple type. If a tuple needs to be changed, it can be converted to an array with the list function. After the change, it can be converted back to a tuple with the tuple function.
•Example: myTuple = (3, 5.8, 'apple')

```
myTuple[1]
```

•Python's tuples can be empty or contain one element.

•ML includes a tuple data type. An ML tuple must have at least two elements.

•The following statement creates a tuple in ML:

```
val myTuple = (3, 5.8, 'apple')
```

•The syntax of a tuple element access is as follows:

```
#1(myTuple);
```

•A new tuple type can be defined in ML with a type declaration, such as the following:

```
type intReal = int * real;
```

Values of this type consist of an integer and a real.

•Lists were first supported in the first functional programming language, Lisp.
•Lists in Scheme and Common Lisp are delimited by parentheses and the elements are not separated by any punctuation. For example: (A B C D)
•Nested lists have the same form. For example: (A (B C) D)
•CAR function returns the first element: (CAR „(A B C))
–This function call returns A
•CDR function returns list except 1st element: (CDR „(A B C))
–This function call returns the list (B C).

•Python includes a list data type (served as Python's arrays). For example:

```
myList = [3, 5.8, "grape"]
x = myList[1]      5.8 is assigned to x
del myList[1]      second element is removed
```

•Python includes a powerful mechanism for creating arrays called list comprehensions. The mechanics of a list comprehension is that a function is applied to each of the elements of a given array and a new array is constructed from the results. The syntax of a Python list comprehension is as follows:

```
[expression for iterate_var in array if condition]
Consider the example: [x * x for x in range(12) if x % 3 == 0]
The range function creates: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
This list comprehension returns: [0, 9, 36, 81]
```

07*Union Types:

•A union is a type whose variables may store different type values at different times during program execution.
•One field of each table entry is for the value of the constant. Suppose that for a particular language being compiled, the types of constants were integer, floating-point, and Boolean.
–In terms of table management, it would be convenient if the same location, a table field, could store a value of any of these three types.
–Then all constant values could be addressed in the same way.
–The type of such a location is, the union of the three value types it can store.

•In C and C++, the union construct is used to specify union structures. The unions in these languages are called free unions (no type checking).

•For the following C union:
–The last assignment is not type checked
–The system cannot determine the current type of the current value of el1,
–It assigns the bit string representation of 27 to the float variable x

```
union flexType {
  int intEl;
  float floatEl;
};
union flexType el1;
float x;
. . .
el1.intEl = 27;
x = el1.floatEl;
```

•A union is declared in F# with a type statement using OR operators (|) to define the components.
Consider the following

```
F# statement:
type intReal =
| IntValue of int
| RealValue of float;;
```

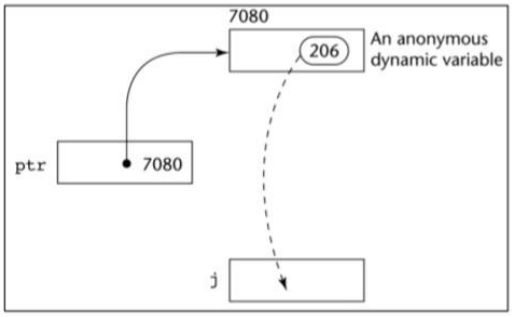
–intReal is the union type.
–IntValue and RealValue are constructors.
–Values of type intReal can be created using the constructors

```
let ir1 = IntValue 17;;
let ir2 = RealValue 3.4;;
```

08*Pointer and Reference Types:

•A pointer type variable has a range of values that consists of memory addresses and a special value
•A pointer provides the power of indirect addressing
•A pointer provides a way to manage dynamic memory
–A pointer can be used to access a location in the area where storage is dynamically created (usually called a heap)

•Pointer Operations:
–Two operations: assignment and dereferencing
–Assignment is used to set a pointer variable's value to some useful address
–Dereferencing yields the value stored at the location represented by the pointer's value
•Dereferencing can be explicit or implicit
•C++ uses an explicit operation via * , example: j = *ptr
–The assignment sets j to the value located at ptr
–The assignment operation j = *ptr
Allocation is sometimes specified with a subprogram, such as malloc in C.
In OOP languages, allocation of heap objects is often specified with the new operator.
C++, which does not provide implicit de-allocation, uses delete as its De-allocation operator.



If ptr is a pointer variable with the value 7080 and the cell whose address is 7080 has the value 206, then the assignment:
j = *ptr sets j to 206.

•Pointers in C and C++:
– Extremely flexible but must be used with great care
– Pointers can point at any variable regardless of when or where it was allocated
–Used for dynamic storage management and addressing
–Pointer arithmetic is possible
–Explicit dereferencing and address-of operators
–Domain type need not be fixed (void *)

•void * can point to any type and can be type checked (cannot be de-referenced)

–In C and C++, the asterisk (*) denotes the dereferencing operation and the ampersand (&) denotes the operator for producing the address of a variable. For example, consider the following code:

```
int *ptr;
int count, init;
. . .
ptr = &init;
count = *ptr;
```

ptr is sets to the address of init count dereferences ptr to produce the value at init init is then assigned to count

–In C and C++, all arrays use zero as the lower bound of their subscript ranges, and array names without subscripts always refer to the address of the first element. Consider the following code:

```
int list[10];
int *ptr;
. . .
ptr = list;
```

The address of list[0] is assigned to ptr*(ptr + 1) is equivalent to list[1] *(ptr + index) is equivalent to list[index]
ptr[index] is equivalent to list[index]

•Reference Types:
–A reference type variable is similar to a pointer, with one important and fundamental difference: A pointer refers to an address in memory, while a reference refers to an object or a value in memory.
–C++ includes a special kind of pointer type (reference type) that is used primarily for formal parameters

•Advantages of both pass-by-reference and pass-by-value
–Java extends C++'s reference variables and allows them to replace pointers entirely

•References are references to objects, rather than being addresses
–C# includes references of Java and pointers of C++

09*Type Checking:

•Type checking is the activity of ensuring that the operands of an operator are of compatible types
–A compatible type is one that is either legal for the operator, or is allowed under language rules to be implicitly converted, by compiler-generated code, to a legal type
–This automatic conversion is called a coercion.

•For example, if an int variable and a float variable are added in Java, the value of the int variable is coerced to float and a floating- point add is done.

•A type error is the application of an operator to an operand of an inappropriate type
–A programming language is strongly typed if type errors are always detected

Chapter 05 : Expressions and Assignment Statements

01*Introduction:

•Expressions are the fundamental means of specifying computations in a programming language
•To understand expression evaluation, we need to be familiar with the orders of operator and operand evaluation
•Essence of imperative languages is dominant role of assignment statements

02*Arithmetic Expressions:

•In programming languages, arithmetic expressions consist of operators, operands, parentheses, and function calls.

•An operator can be unary, meaning it has a single operand, binary, meaning it has two operands, or ternary, meaning it has three operands.

•In most programming languages, binary operators are infix, which means they appear between their operands.

•One exception is Perl, which has some operators that are prefix, which means they precede their operands.

•Operator Evaluation Order:

–Precedence:
•The value of an expression depends at least in part on the order of evaluation of the operators in the expression.
•Example: a + b * c → Assume a = 3, b = 4, c = 5
–If evaluated from left to right, the answer is 35
–If evaluated from right to left, the answer is 23
•However, it is necessary to follow a precedence rule in order to get reasonable result
•The operator precedence rules for expression evaluation partially define the order in which the operators of different precedence levels are evaluated.
•The operator precedence rules of the common imperative languages are nearly all the same, because they are based on those of mathematics.
•Many languages also include unary versions of addition and subtraction
•In all of the common imperative languages, the unary minus operator can appear in an expression either at the beginning or anywhere inside the expression.
•Example: a + (- b) * c
a + - b * c //illegal
- a / b
- a * b
- a * * b

•The precedence of the arithmetic operators are as follows:

	Ruby	*C-Based Languages*
Highest	**	postfix ++, --
	unary +, -	prefix ++, --, unary +, -
	*, /, %	*, /, %
Lowest	binary +, -	binary +, -

–Associativity:

•Associativity in common languages is left to right.

–Example: a + b – c. Because the addition and subtraction are of the same level of precedence, addition is evaluated first due to left associativity.

•Exponentiation in Fortran and Ruby is right associative

–Example: A ** B ** C. The right operator is evaluated first

•In APL, operators have the same level of precedence.

•The order of evaluation of operators in APL expressions is determined entirely by the associativity rule, which is right to left for all operators.

–Example: Assume that A = 3, B = 4, C = 5

```
A x B + C = 3 x 4 + 5 = 3 x 9 = 27
C + A x B = 5 + 3 x 4 = 5 + 12 = 17
```

– Parentheses:

•Programmers can alter the precedence and associativity rules by placing parentheses in expressions.

– Example: Assume that A = 3, B = 4, C = 5, D = 1

```
(A + B) * C = (3 + 4) * 5 = 7 * 5 = 35
(A + B) + (C + D) = (3 + 4) + (5 + 1) = 7 + 6 = 13
```

–Conditional Expressions:

•C-based languages (e.g., C, C++)

•An example:

```
average = (count == 0)? 0 : sum / count
```

Evaluates as if written like

```
if (count == 0) average = 0
else average = sum /count
```

–Side Effects:

•Consider the following expression with a global variable

```
a + fun (a) // a is global
```

If fun does not change the value of a, the addition has no side effect in changing the value of the expression. If fun changes the value of a, the value of the expression is effected.

•Consider the following code:

Starting with the main:

```
a = a + fun1();
```

if left associativity

```
a = 5 + 3 = 8
```

if right associativity

```
a = 17 + 3 = 20
int a = 5;//global
int fun1() {
a = 17;
return 3;
} // end fun1
void main() {
a = a + fun1();
}// end main
```

03*Overloaded Operators:

•Use of an operator for more than one purpose is called operator overloading
–Example 1: '+' is used for addition for integers and
floating-point numbers
–Example 2: In Java, '+' is used for addition in arithmetic expressions and string catenation.
–Example 3: In C++, ampersand '&' is used as a bitwise logical AND operator and as a unary operator. (totally different and can be confusing)

Type Conversions:

•A narrowing conversion is one that converts an object to a type that cannot include all of the values of the original type e.g., float to int
•A widening conversion is one in which an object is converted to a type that can include at least approximations to all of the values of the original type e.g., int to float
•Called casting in C-based languages
–C: (int) angle
–Ada: Float(Sum)
Note that Ada's syntax is similar to that of function calls

•Explicit Type Conversion:

–In the C-based languages, explicit type conversions are called casts.
•C: (int) angle
•Ada: float(sum)
-Note that Ada's syntax is similar to that of function calls

•Relational Expressions:

–Use relational operators and operands of various types
–Evaluate to some Boolean representation
–Operator symbols used vary somewhat among languages (!=, /=, .NE., <>, #)
–JavaScript and PHP have two additional relational
operators: === and !== similar to their relatives == and
!= except that they do not coerce their operands.
"7" == 7 is true but "7" === 7 is false

•Boolean Expressions:
– Operands are Boolean and the result is Boolean as well
– Example of boolean operators

FORTRAN 77	FORTRAN 90	C	Ada
.AND.	and	&&	and
.OR.	or	||	or
.NOT.	not	!	not
			xor

– No Boolean type in C
– The expressions a < b < c is legal in C (left associativity)
Short-Circuit Evaluation:
•An expression in which the result is determined without evaluating all of the operands and/or operators

```
(13 * a) * (b / 13 – 1)
If a = 0, there is no need to evaluate (b / 13 - 1)
```

• Problem with non-short-circuit evaluation

```
index = 1;
while (index <= length) && (LIST[index] != value)
index++;
```

–When index = length, LIST [index] will cause an indexing problem (assuming LIST has length - 1 elements)

•C, C++, and Java: use short-circuit evaluation for the usual Boolean operators (&& and ||), but also provide bitwise Boolean operators (& and |) that are not short-circuit
•Short-circuit evaluation exposes the potential problem of side effects in expressions
e.g. (a > b) || (b++ / 3)

04*Assignment Statements:

•Simple Assignments:
–The general syntax

```
<target_var> <assign_operator> <expression>
```

– The assignment operator
= FORTRAN, BASIC, the C-based languages := ALGOLs, Pascal, Ada
– = can be bad when it is overloaded for the relational operator for equality (that's why the C-based languages use == as the relational operator)

•Conditional Targets:
– In Perl

```
($flag ? $total : $subtotal) = 0
```

Which is equivalent to

```
if ($flag){
$total = 0
} else {
$subtotal = 0
}
```

•Compound Assignment Operators:
–A shorthand method of specifying a commonly needed form of assignment
–Introduced in ALGOL; adopted by C
–Example:

```
a = a + b
```

is written as

```
a += b
```

•Unary Assignment Operators:
–In C-based languages, they combine increment and decrement operations with assignment
–Examples:

```
sum = ++count (count incremented, assigned to sum)
sum = count++ (count assigned to sum, then incremented)
count++ (count incremented)
++count (count incremented)
```

•Assignment as an Expression:
–In C, C++, and Java, the assignment statement produces a result and can be used as operands
–Example:

```
while ((ch = getchar())!= EOF){…}
```

ch = getchar() is carried out; the result (assigned to ch) is used as a conditional value for the while statement

•Multiple Assignments:
–Perl, Ruby, and Lua, provide multiple-target, multiple-source assignment statements.
–In Perl, Example: ($first, $second, $third) = (20, 40, 60);
–If the values of two variables must be interchanged ($first, $second) = ($second, $first)

Chapter 06 : Statement-Level Control Structures

01*Introduction:

•Computations in imperative-language programs are accomplished by evaluating expressions and assigning the resulting values to variables.
•Two additional linguistic mechanisms are necessary to make the computations in programs flexible and powerful: some means of selecting among alternative control flow paths (of statement execution) and some means of causing the repeated execution of statements or sequences of statements (Control Statements).
•Computations in functional programming languages are accomplished by evaluating expressions and applying functions to given parameters. Furthermore, the flow of execution among the expressions and functions is controlled by other expressions and functions, although some of them are similar to the control statements in the imperative languages.
•A control structure is a control statement and the collection of statements whose execution it controls.

02*Selection Statements:

•Two-Way Selection Statements:

–General Form:

```
if <control-expression>
    then clause
    else clause
```

This means that if control-expression is true, the 'then' clause will be executed; otherwise, the 'else' clause will be executed.

–Control Expression

•Control expressions are specified in parentheses if the thenreserved word (or some other syntactic marker) is not used to introduce the then clause

•Ruby use then keyword

•In C89, C99, Python, and C++, the control expression can be arithmetic

•In languages such as Ada, Java, Ruby, and C#, the control expression must be Boolean

–Clause Form

•In many contemporary languages, the then and else clauses can be single statements or compound statements

•In Perl, all clauses must be delimited by braces (they must be compound)

•In Fortran 95, Ada, and Ruby, clauses are statement sequences

•Python uses indentation to define clause

```
if x > y :
x = y
print "case 1"
```

–Nesting Selectors (1/2):

•Consider the Java example:

•The else clause matched the first or second then clause?

–The indentation seems to indicate that the else clause belongs with the first then clause (indentation in Python and F#)

–In other programming languages, indentation has no effect onsemantics and ignored by their compilers.

```
if (sum == 0)
  if (count == 0)
       result = 0;
else result = 1;
```

C-based languages:

```
if (sum == 0)
 if (count == 0)
   result = 0;
else result = 1;
Equivalent to
if (sum == 0) {
   if (count == 0) {
        result = 0;
      }
} else {
    result = 1;
 }
```

Ruby and Perl languages:

```
if a > b then sum = sum + a
       acount = acount + 1
else sum = sum + b
     bcount = bcount + 1
end
```

Nested in Ruby:

```
if sum = 0 then
   if count = 0 then
      result = 0
   end
else
     result = 1
end
```

–Selector Expressions:

•In the functional languages ML, F#, and LISP, the selector is not a statement; it is an expression that results in a value.

•Consider the F# example:

•This creates the name y and sets it to either x or 2 * x, depending on whether x is greater than zero.

• If no else clause, it returns a special type with no value.

```
Let y = if x > 0 then x
  else 2 * x;;
```

•Multiple-Selection Statements:

–Allow the selection of one of any number of statements or statement groups

–General Form

```
switch (expression) {
case const_expr_1: stmt_1;
. . .
case const_expr_n: stmt_n;
[default: stmt_n+1]
  }
```

–Java Example:

```
switch (i) {
case 1:
case 3: odd += 1;
            sumOdd += i;
            break;
case 2: case 4: even += 1;
            sumeven += i;
            break;
default: printf("Error index: %d\n", i);
}
```

–The presence or absence of break statement in the different cases of the code has a huge effect on the output

–C Example:

```
switch (x)
    default:
    if (prime(x))
       case 2: case 3: case 5: case 7:
       process_prime(x);
    else
       case 4: case 6: case 8: case 9:
       process_composite(x);
```

–The C switch statement has virtually no restrictions on the placement of the case expressions, which are treated as if they were normal statement labels.

–C# Example:

```
switch (value) {
  case -1:
     Negatives++;
     break;
  case 0:
     Zeros++;
     goto case 1;
  case 1:
     Positives++;
     break;
  default:
     Console.WriteLine("Error in switch \n");
     break;
  }
```

– Every selectable segment must end with an explicit unconditional branch statement (break or goto statement).

–Ruby Example:

```
 leap = case
   when year % 400 == 0 then true
   when year % 100 == 0 then false
   else year % 4 == 0
   end
```

–This case expression evaluates to true if year is a leap year.
–The value of the case expression is the value of the first then expression whose Boolean expression is true.

–Multiple Selection Using if:

```
    if E1 goto 1
    if E2 goto 2
    . . .
1: S1
    goto out
2: S2
    goto out
. . .
out: . . .
```

```
Example in Python:

if count < 10 :
    bag1 = True
elif count < 100 :
    bag2 = True
elif count < 1000 :
    bag3 = True
```

03*Iteration Statements:

•An iterative statement (Loop) is one that causes a statement or collection of statements to be executed zero, one, or more times.
–The body of an iterative statement is the collection of statements whose execution is controlled by the iteration statement.
–The term pretest means that the test for loop completion occurs before the loop body is executed
–The term posttest means that it occurs after the loop body is executed.

–A counting iterative control statement has a variable, called the loop variable, in which the count value is maintained.
–It also includes the loop parameters:
•The initial value --> he start value of the loop
•The terminal value --> the condition where the loop stops
•The stepsize --> the difference between sequential variable values.
•Counter-Controlled Loops
–General form of C's for statement

```
for(expression_1; expression_2; expression_3)
  loop body
```

Operational semantics description of the for statement:

```
  expression_1
loop:
  if expression_2 goto out
  [loop body]
  expression_3
  goto loop
out: . . .
```

04*Counter-Controlled Loops:

– C Example:

```
for (count1 = 0, count2 = 1.0; count1 <= 10 && count2 <= 100.0; sum = ++count1 +count2, count2 *= 2.5);
  count1 = 0
  count2 = 1.0
loop:
  if count1 > 10 goto out
  if count2 > 100.0 goto out
  count1 = count1 + 1
  sum = count1 + count2
  count2 = count2 * 2.5
  goto loop
out: . . .
```

–Python for loop structure and Example:

```
for loop_variable in object:
- loop body
for count in [2, 4, 6]:
 print count
```

Produces

```
2
4
6
```

–Simple counting loops in Python using range function

range(5) --> returns [0, 1, 2, 3, 4]

range(2, 7) --> returns [2, 3, 4, 5, 6]

range(0, 8, 2) --> returns [0, 2, 4, 6]

–Loop structure in functional languages such as F#

```
let rec forLoop loopBoody reps =
    if reps <= 0 then
        ()
   else
     loopBody()
     forLoop loopBody, (reps – 1);;
```

– Statements repetition based on a Boolean expression rather than a counter.

```
while (control_expression)
  loop body
do
 loop body
while (control_expression);
```

05*Logically Controlled Loops:

–C# statements example code:

```
while loop:
if control_expr is false goto out
 [loop body]
 goto loop
out: . . .
do-while loop:
 [loop body]
 if control_expr is true goto loop
```

– Loop structure in functional languages such as F#

```
let rec whileLoop test body =
 if test() then
   body
   whileLoop test body
 else
   ();;
```

07*User-Located Loop Control Mechanisms:

–C, C++, Python, Ruby, and C# have unconditional unlabeled exits (break).

–Java and Perl have unconditional labeled exits (break in Java, last in Perl).

–Java Example of nested loops

```
outerLoop:
for (row = 0; row < numRows; row++)
 for (col = 0; col < numCols; col++) {
   sum += mat[row][col];
   if (sum > 1000.0)
     break outerLoop;
}
```

–Control statements continue and break

```
while (sum < 1000) {
   getnext(value);
   if (value < 0)
      continue;
   sum += value;
}
```

```
while (sum < 1000) {
   getnext(value);
   if (value < 0)
      break;
   sum += value;
}
```